10 PROGRAMMING PROBLEMS

Problem Statements • Algorithms • Flowcharts • Python Code
Sample Inputs • Sample Outputs • Mermaid Flowchart Source

This document contains complete Python 3 solutions for all ten topics. Each section includes a problem statement, step-by-step algorithm, a visual flowchart, sample input/output, runnable code, and Mermaid source code for the flowchart.

Topics included	Topics included
1. University Scholarship Allocation	6. Railway Fare and Reservation Calculator
2. Progressive Electricity Billing	7. Retail Billing with Multiple Discount Rules
3. Hospital Billing and Patient Classification	8. Manufacturing Cost and Profitability Analysis
4. Employee Payroll and Deduction Analysis	9. Multi-Currency Travel Budget Analysis
5. Multi-Plan Telecommunications Cost Optimization	10. Campus Event Financial Planning

Note: All fee rules, prices, discount thresholds, and exchange rates are illustrative examples for learning. Update them if your assignment specifies different rules.

1. University Scholarship Allocation

Problem Statement

Write a program to calculate a student's scholarship percentage and payable tuition fee using marks and annual family income.

Algorithm

1. Start and read student name, marks (out of 100), family income, and tuition fee.

2. If marks ≥ 90 and income ≤ ■300,000, set scholarship rate to 100%.

3. Else if marks ≥ 80 and income ≤ ■500,000, set scholarship rate to 50%.

4. Else if marks ≥ 70 and income ≤ ■700,000, set scholarship rate to 25%; otherwise set it to 0%.

5. Calculate scholarship amount = tuition fee × rate / 100.

6. Calculate payable fee = tuition fee – scholarship amount.

7. Display scholarship rate, scholarship amount, and payable fee. Stop.

Flowchart (visual outline)

Start ↓ Input student, marks, income, fee ↓ Check scholarship eligibility ↓ Set rate: 100%, 50%, 25%, or 0% ↓ Calculate scholarship and payable fee ↓ Display result ↓ End

Sample Input

```
Ananya
92
280000
80000
```

Sample Output

```
Student: Ananya
Marks: 92
Family income: Rs. 280000.00
Tuition fee: Rs. 80000.00
Scholarship rate: 100%
Scholarship amount: Rs. 80000.00
Payable fee: Rs. 0.00
```

Python 3 Code

```
student = input("Student name: ")
marks = float(input("Marks out of 100: "))
income = float(input("Annual family income: Rs. "))
fee = float(input("Tuition fee: Rs. "))

if marks >= 90 and income <= 300000:
    rate = 100
elif marks >= 80 and income <= 500000:
    rate = 50
elif marks >= 70 and income <= 700000:
    rate = 25
else:
    rate = 0

scholarship = fee * rate / 100
payable = fee - scholarship

print(f"Student: {student}")
print(f"Marks: {marks:g}")
print(f"Family income: Rs. {income:.2f}")
print(f"Tuition fee: Rs. {fee:.2f}")
print(f"Scholarship rate: {rate}%")
print(f"Scholarship amount: Rs. {scholarship:.2f}")
print(f"Payable fee: Rs. {payable:.2f}")
```

Mermaid Flowchart Code

```
flowchart TD
    A([Start]) --> B[/Input student, marks, income, fee/]
    B --> C{Marks >= 90 AND income <= 300000?}
    C -- Yes --> D[Rate = 100%]
    C -- No --> E{Marks >= 80 AND income <= 500000?}
    E -- Yes --> F[Rate = 50%]
    E -- No --> G{Marks >= 70 AND income <= 700000?}
    G -- Yes --> H[Rate = 25%]
    G -- No --> I[Rate = 0%]
    D --> J[Calculate scholarship and payable fee]
    F --> J
    H --> J
    I --> J
    J --> K[/Display result/]
    K --> L([End])
```

2. Progressive Electricity Billing

Problem Statement

Write a program to calculate an electricity bill using progressive unit slabs and a fixed charge. Rates used: first 100 units at Rs. 1.50/unit, next 100 at Rs. 2.50/unit, and units above 200 at Rs. 4.00/unit.

Algorithm

1. Read consumer name, units consumed, and fixed charge.

2. If units ≤ 100, energy charge = units × 1.50.

3. Else if units ≤ 200, energy charge = 150 + (units – 100) × 2.50.

4. Otherwise, energy charge = 400 + (units – 200) × 4.00.

5. Total bill = energy charge + fixed charge.

6. Display energy charge and total bill. Stop.

Flowchart (visual outline)

Start ↓ Input units and fixed charge ↓ Units ≤ 100? ↓ Yes: units × 1.50 ↓ No: Units ≤ 200? ↓ Yes: 150 + excess × 2.50 ↓ No: 400 + excess × 4.00 ↓ Add fixed charge ↓ Display bill ↓ End

Sample Input

```
Ravi
250
50
```

Sample Output

```
Consumer: Ravi
Units consumed: 250
Energy charge: Rs. 600.00
Fixed charge: Rs. 50.00
Total bill: Rs. 650.00
```

Python 3 Code

```
consumer = input("Consumer name: ")
units = float(input("Units consumed: "))
fixed = float(input("Fixed charge: Rs. "))

if units <= 100:
    energy = units * 1.50
elif units <= 200:
    energy = 150 + (units - 100) * 2.50
else:
    energy = 400 + (units - 200) * 4.00

total = energy + fixed
print(f"Consumer: {consumer}")
print(f"Units consumed: {units:g}")
print(f"Energy charge: Rs. {energy:.2f}")
print(f"Fixed charge: Rs. {fixed:.2f}")
print(f"Total bill: Rs. {total:.2f}")
```

Mermaid Flowchart Code

```
flowchart TD
    A([Start]) --> B[/Input consumer, units, fixed charge/]
    B --> C{Units <= 100?}
    C -- Yes --> D[Energy = units * 1.50]
    C -- No --> E{Units <= 200?}
    E -- Yes --> F[Energy = 150 + (units - 100) * 2.50]
    E -- No --> G[Energy = 400 + (units - 200) * 4.00]
    D --> H[Total = energy + fixed charge]
    F --> H
    G --> H
    H --> I[/Display bill/]
    I --> J([End])
```

3. Hospital Billing and Patient Classification

Problem Statement

Write a program to classify a patient as Child, Adult, or Senior based on age and calculate the final hospital bill. Senior patients receive a 10% discount.

Algorithm

1. Read patient name, age, days admitted, daily room charge, test charges, and treatment charges.

2. If age < 13, classification = Child; else if age ≥ 60, classification = Senior; otherwise Adult.

3. Room cost = days admitted × daily room charge.

4. Subtotal = room cost + test charges + treatment charges.

5. If classification is Senior, discount = 10% of subtotal; otherwise discount = 0.

6. Final bill = subtotal – discount.

7. Display classification, subtotal, discount, and final bill. Stop.

Flowchart (visual outline)

Start ↓ Input patient details and charges ↓ Classify by age ↓ Calculate room cost and subtotal ↓ Senior patient? ↓ Yes: apply 10% discount; No: no discount ↓ Calculate final bill ↓ Display result ↓ End

Sample Input

```
Kumar
65
3
2000
1500
8000
```

Sample Output

```
Patient: Kumar
Age: 65
Classification: Senior
Room cost: Rs. 6000.00
Subtotal: Rs. 15500.00
Discount: Rs. 1550.00
Final bill: Rs. 13950.00
```

Python 3 Code

```
name = input("Patient name: ")
age = int(input("Age: "))
days = int(input("Days admitted: "))
room_rate = float(input("Daily room charge: Rs. "))
tests = float(input("Test charges: Rs. "))
treatment = float(input("Treatment charges: Rs. "))

if age < 13:
    category = "Child"
elif age >= 60:
    category = "Senior"
else:
    category = "Adult"

room_cost = days * room_rate
subtotal = room_cost + tests + treatment
discount = subtotal * 0.10 if category == "Senior" else 0
final_bill = subtotal - discount

print(f"Patient: {name}")
print(f"Age: {age}")
print(f"Classification: {category}")
print(f"Room cost: Rs. {room_cost:.2f}")
print(f"Subtotal: Rs. {subtotal:.2f}")
print(f"Discount: Rs. {discount:.2f}")
print(f"Final bill: Rs. {final_bill:.2f}")
```

Mermaid Flowchart Code

```
flowchart TD
    A([Start]) --> B[/Input patient details and charges/]
    B --> C{Age < 13?}
    C -- Yes --> D[Category = Child]
    C -- No --> E{Age >= 60?}
    E -- Yes --> F[Category = Senior]
    E -- No --> G[Category = Adult]
    D --> H[Calculate room cost and subtotal]
    F --> H
    G --> H
    H --> I{Senior?}
```

```
I -- Yes --> J[Discount = 10% of subtotal]
I -- No --> K[Discount = 0]
J --> L[Final bill = subtotal - discount]
K --> L
L --> M[/Display category and bill/]
M --> N([End])
```

4. Employee Payroll and Deduction Analysis

Problem Statement

Write a program to calculate an employee's gross salary, income tax, total deductions, and net salary. The tax rate is entered as a percentage.

Algorithm

1. Read employee name, basic salary, HRA, other allowances, tax rate, and other deductions.

2. Gross salary = basic salary + HRA + other allowances.

3. Income tax = gross salary × tax rate / 100.

4. Total deductions = income tax + other deductions.

5. Net salary = gross salary – total deductions.

6. Display gross salary, tax, total deductions, and net salary. Stop.

Flowchart (visual outline)

Start ↓ Input salary components and tax rate ↓ Calculate gross salary ↓ Calculate income tax ↓ Add other deductions ↓ Calculate net salary ↓ Display payroll summary ↓ End

Sample Input

```
Priya
30000
8000
2000
10
1000
```

Sample Output

```
Employee: Priya
Gross salary: Rs. 40000.00
Income tax: Rs. 4000.00
Other deductions: Rs. 1000.00
Total deductions: Rs. 5000.00
Net salary: Rs. 35000.00
```

Python 3 Code

```
name = input("Employee name: ")
basic = float(input("Basic salary: Rs. "))
hra = float(input("HRA: Rs. "))
allowances = float(input("Other allowances: Rs. "))
tax_rate = float(input("Tax rate (%): "))
other_deductions = float(input("Other deductions: Rs. "))

gross = basic + hra + allowances
tax = gross * tax_rate / 100
total_deductions = tax + other_deductions
net = gross - total_deductions

print(f"Employee: {name}")
print(f"Gross salary: Rs. {gross:.2f}")
print(f"Income tax: Rs. {tax:.2f}")
print(f"Other deductions: Rs. {other_deductions:.2f}")
print(f"Total deductions: Rs. {total_deductions:.2f}")
print(f"Net salary: Rs. {net:.2f}")
```

Mermaid Flowchart Code

```
flowchart TD
    A([Start]) --> B[/Input salary components and tax rate/]
    B --> C[Gross = basic + HRA + allowances]
    C --> D[Tax = gross * tax rate / 100]
    D --> E[Total deductions = tax + other deductions]
    E --> F[Net salary = gross - deductions]
    F --> G[/Display payroll summary/]
    G --> H([End])
```

5. Multi-Plan Telecommunications Cost Optimization

Problem Statement

Write a program to compare three mobile plans and recommend the lowest-cost plan for the customer's monthly call, data, and SMS usage. Plan rates are illustrative.

Algorithm

1. Read monthly call minutes, data usage in GB, and SMS count.

2. Plan A: base Rs. 299 includes 300 minutes, 2 GB, and 100 SMS; excess rates Rs. 1/min, Rs. 50/GB, Rs. 0.25/SMS.

3. Plan B: base Rs. 499 includes 800 minutes, 5 GB, and 300 SMS; excess rates Rs. 0.75/min, Rs. 40/GB, Rs. 0.20/SMS.

4. Plan C: base Rs. 699 includes 1500 minutes, 10 GB, and 500 SMS; excess rates Rs. 0.50/min, Rs. 30/GB, Rs. 0.10/SMS.

5. For each plan, add the base fee and charges for usage exceeding included limits.

6. Choose the plan with the minimum total cost.

7. Display all plan costs and the recommended plan. Stop.

Flowchart (visual outline)

Start ↓ Input minutes, data, and SMS ↓ Calculate Plan A cost ↓ Calculate Plan B cost ↓ Calculate Plan C cost ↓ Select minimum cost ↓ Display all costs and recommendation ↓ End

Sample Input

```
600
4
200
```

Sample Output

```
Monthly usage: 600 minutes, 4 GB, 200 SMS
Plan A cost: Rs. 724.00
Plan B cost: Rs. 499.00
Plan C cost: Rs. 699.00
Recommended plan: Plan B
```

Python 3 Code

```
minutes = float(input("Monthly call minutes: "))
data = float(input("Data usage (GB): "))
sms = float(input("SMS count: "))

plans = {
    "Plan A": (299, 300, 2, 100, 1.00, 50.00, 0.25),
    "Plan B": (499, 800, 5, 300, 0.75, 40.00, 0.20),
    "Plan C": (699, 1500, 10, 500, 0.50, 30.00, 0.10),
}

costs = {}
for name, p in plans.items():
    base, inc_min, inc_data, inc_sms, r_min, r_data, r_sms = p
    extra_min = max(0, minutes - inc_min) * r_min
    extra_data = max(0, data - inc_data) * r_data
    extra_sms = max(0, sms - inc_sms) * r_sms
    costs[name] = base + extra_min + extra_data + extra_sms

for name, cost in costs.items():
    print(f"{name} cost: Rs. {cost:.2f}")

best = min(costs, key=costs.get)
print(f"Recommended plan: {best}")
```

Mermaid Flowchart Code

```
flowchart TD
    A([Start]) --> B[/Input minutes, data, SMS/]
    B --> C[Calculate Plan A total cost]
    C --> D[Calculate Plan B total cost]
    D --> E[Calculate Plan C total cost]
    E --> F[Find minimum of the three costs]
    F --> G[/Display costs and recommended plan/]
    G --> H([End])
```

6. Railway Fare and Reservation Calculator

Problem Statement

Write a program to calculate railway fare based on distance and passenger category, and confirm a reservation only if enough seats are available. Adult fare is Rs. 1.20/km; children receive 50% off and seniors receive 20% off.

Algorithm

1. Read passenger name, distance, category, available seats, and requested seats.

2. Base fare per passenger = distance × 1.20.

3. If category is Child, discount rate = 50%; if Senior, 20%; otherwise 0%.

4. Fare per passenger = base fare × (1 – discount rate / 100).

5. If requested seats ≤ available seats, confirm booking and calculate remaining seats; otherwise reject booking.

6. Total fare = fare per passenger × requested seats when confirmed.

7. Display booking status, total fare, and remaining seats. Stop.

Flowchart (visual outline)

Start ↓ Input distance, category, and seats ↓ Calculate base fare ↓ Apply category discount ↓ Enough seats available? ↓ Yes: confirm booking; No: reject booking ↓ Display status and fare ↓ End

Sample Input

```
Meena
250
Senior
10
2
```

Sample Output

```
Passenger: Meena
Category: Senior
Fare per passenger: Rs. 240.00
Booking status: Confirmed
Seats booked: 2
Total fare: Rs. 480.00
Seats remaining: 8
```

Python 3 Code

```
name = input("Passenger name: ")
distance = float(input("Distance (km): "))
category = input("Category (Child/Adult/Senior): ").strip().lower()
available = int(input("Available seats: "))
requested = int(input("Seats requested: "))

base_fare = distance * 1.20
if category == "child":
    discount = 0.50
elif category == "senior":
    discount = 0.20
else:
    discount = 0.0

fare = base_fare * (1 - discount)

print(f"Passenger: {name}")
print(f"Category: {category.title()}")
print(f"Fare per passenger: Rs. {fare:.2f}")
if requested <= available:
    print("Booking status: Confirmed")
    print(f"Seats booked: {requested}")
    print(f"Total fare: Rs. {fare * requested:.2f}")
    print(f"Seats remaining: {available - requested}")
else:
    print("Booking status: Not available")
    print("Total fare: Rs. 0.00")
    print(f"Seats remaining: {available}")
```

Mermaid Flowchart Code

```
flowchart TD
    A([Start]) --> B[/Input distance, category, seats/]
    B --> C[Calculate base fare]
    C --> D[Apply category discount]
    D --> E{Requested seats <= available seats?}
    E -- Yes --> F[Confirm booking and calculate total fare]
    E -- No --> G[Reject booking]
    F --> H[/Display status, fare, remaining seats/]
    G --> H
    H --> I([End])
```

7. Retail Billing with Multiple Discount Rules

Problem Statement

Write a program to calculate a retail bill. Purchases of Rs. 10,000 or more receive 10% off; purchases of Rs. 5,000 or more receive 5% off. Gold members receive an additional 5% discount after the purchase discount. Tax is 5% of the discounted amount.

Algorithm

1. Read customer name, purchase amount, and membership type.

2. If amount ≥ Rs. 10,000, purchase discount = 10%; else if amount ≥ Rs. 5,000, discount = 5%; otherwise 0%.

3. Subtract purchase discount from the amount.

4. If membership is Gold, apply an additional 5% discount to the reduced amount.

5. Tax = 5% of the amount after both discounts.

6. Final bill = amount after discounts + tax.

7. Display discounts, tax, and final bill. Stop.

Flowchart (visual outline)

Start ↓ Input purchase amount and membership ↓ Choose purchase discount ↓ Apply extra 5% if Gold member ↓ Calculate tax on discounted amount ↓ Calculate final bill ↓ Display bill ↓ End

Sample Input

```
Arjun
12000
Gold
```

Sample Output

```
Customer: Arjun
Purchase amount: Rs. 12000.00
Purchase discount: Rs. 1200.00
Gold discount: Rs. 540.00
Tax: Rs. 513.00
Final bill: Rs. 10773.00
```

Python 3 Code

```
name = input("Customer name: ")
amount = float(input("Purchase amount: Rs. "))
membership = input("Membership (Regular/Gold): ").strip().lower()

if amount >= 10000:
    purchase_discount = amount * 0.10
elif amount >= 5000:
    purchase_discount = amount * 0.05
else:
    purchase_discount = 0.0

after_purchase_discount = amount - purchase_discount
gold_discount = (after_purchase_discount * 0.05
                 if membership == "gold" else 0.0)
taxable = after_purchase_discount - gold_discount
tax = taxable * 0.05
final_bill = taxable + tax

print(f"Customer: {name}")
print(f"Purchase amount: Rs. {amount:.2f}")
print(f"Purchase discount: Rs. {purchase_discount:.2f}")
print(f"Gold discount: Rs. {gold_discount:.2f}")
print(f"Tax: Rs. {tax:.2f}")
print(f"Final bill: Rs. {final_bill:.2f}")
```

Mermaid Flowchart Code

```
flowchart TD
    A([Start]) --> B[/Input amount and membership/]
    B --> C{Amount >= 10000?}
    C -- Yes --> D[Purchase discount = 10%]
    C -- No --> E{Amount >= 5000?}
    E -- Yes --> F[Purchase discount = 5%]
    E -- No --> G[Purchase discount = 0%]
    D --> H{Gold member?}
    F --> H
    G --> H
    H -- Yes --> I[Apply extra 5% discount]
    H -- No --> J[No membership discount]
    I --> K[Calculate tax and final bill]
    J --> K
    K --> L[/Display bill/]
    L --> M([End])
```

8. Manufacturing Cost and Profitability Analysis

Problem Statement

Write a program to calculate total manufacturing cost, cost per unit, sales revenue, and profit or loss.

Algorithm

1. Read units produced, material cost, labour cost, overhead cost, units sold, and selling price per unit.

2. If units produced is 0, display an invalid production quantity message and stop.

3. Total cost = material + labour + overhead.

4. Cost per unit = total cost / units produced.

5. Revenue = units sold × selling price per unit.

6. Profit = revenue – total cost.

7. If profit > 0, status = Profit; if profit < 0, status = Loss; otherwise status = Break-even.

8. Display total cost, unit cost, revenue, profit/loss, and status. Stop.

Flowchart (visual outline)

Start ↓ Input production, costs, and sales ↓ Production quantity > 0? ↓ No: display invalid quantity ↓ Yes: calculate total cost and unit cost ↓ Calculate revenue and profit ↓ Classify as profit, loss, or break-even ↓ Display analysis ↓ End

Sample Input

```
1000
40000
20000
10000
1000
90
```

Sample Output

```
Units produced: 1000
Total cost: Rs. 70000.00
Cost per unit: Rs. 70.00
Revenue: Rs. 90000.00
Profit/Loss: Rs. 20000.00
Status: PROFIT
```

Python 3 Code

```
produced = int(input("Units produced: "))
material = float(input("Material cost: Rs. "))
labour = float(input("Labour cost: Rs. "))
overhead = float(input("Overhead cost: Rs. "))
sold = int(input("Units sold: "))
selling_price = float(input("Selling price per unit: Rs. "))

if produced <= 0:
    print("Invalid production quantity")
else:
    total_cost = material + labour + overhead
    cost_per_unit = total_cost / produced
    revenue = sold * selling_price
    profit = revenue - total_cost

    if profit > 0:
        status = "PROFIT"
    elif profit < 0:
        status = "LOSS"
    else:
        status = "BREAK-EVEN"

    print(f"Units produced: {produced}")
    print(f"Total cost: Rs. {total_cost:.2f}")
    print(f"Cost per unit: Rs. {cost_per_unit:.2f}")
    print(f"Revenue: Rs. {revenue:.2f}")
    print(f"Profit/Loss: Rs. {profit:.2f}")
    print(f"Status: {status}")
```

Mermaid Flowchart Code

```
flowchart TD
    A([Start]) --> B[/Input production, costs, sales/]
    B --> C{Units produced > 0?}
    C -- No --> D[/Display invalid quantity/]
    C -- Yes --> E[Calculate total cost and cost per unit]
    E --> F[Calculate revenue and profit]
    F --> G{Profit > 0?}
    G -- Yes --> H[Status = PROFIT]
    G -- No --> I{Profit < 0?}
```

```
I -- Yes --> J[Status = LOSS]
I -- No --> K[Status = BREAK-EVEN]
H --> L[/Display analysis/]
J --> L
K --> L
D --> M([End])
L --> M
```

9. Multi-Currency Travel Budget Analysis

Problem Statement

Write a program to convert travel expenses into Indian rupees and check whether total expenses stay within the budget. Exchange rates are sample values supplied as inputs.

Algorithm

1. Read the total budget in INR.

2. Read expense amount and exchange rate to INR for each expense.

3. Convert each expense: INR cost = amount × exchange rate.

4. Add converted expenses to calculate total expenses.

5. Remaining budget = budget – total expenses.

6. If remaining budget ≥ 0, status = Within budget; otherwise status = Over budget.

7. Display converted expenses, total, remaining budget, and status. Stop.

Flowchart (visual outline)

Start ↓ Input budget, expenses, and exchange rates ↓ Convert each expense to INR ↓ Calculate total expenses ↓ Calculate remaining budget ↓ Within budget? ↓ Yes: within budget; No: over budget ↓ Display summary ↓ End

Sample Input

```
100000
300
83
200
90
100
105
```

Sample Output

```
Budget: Rs. 100000.00
Hotel converted: Rs. 24900.00
Food converted: Rs. 18000.00
Transport converted: Rs. 10500.00
Total expenses: Rs. 53400.00
Remaining budget: Rs. 46600.00
Status: WITHIN BUDGET
```

Python 3 Code

```
budget = float(input("Travel budget in INR: Rs. "))
hotel_amount = float(input("Hotel amount in foreign currency: "))
hotel_rate = float(input("Hotel currency rate to INR: "))
food_amount = float(input("Food amount in foreign currency: "))
food_rate = float(input("Food currency rate to INR: "))
transport_amount = float(input("Transport amount in foreign currency: "))
transport_rate = float(input("Transport currency rate to INR: "))

hotel = hotel_amount * hotel_rate
food = food_amount * food_rate
transport = transport_amount * transport_rate
total = hotel + food + transport
remaining = budget - total
status = "WITHIN BUDGET" if remaining >= 0 else "OVER BUDGET"

print(f"Budget: Rs. {budget:.2f}")
print(f"Hotel converted: Rs. {hotel:.2f}")
print(f"Food converted: Rs. {food:.2f}")
print(f"Transport converted: Rs. {transport:.2f}")
print(f"Total expenses: Rs. {total:.2f}")
print(f"Remaining budget: Rs. {remaining:.2f}")
print(f"Status: {status}")
```

Mermaid Flowchart Code

```
flowchart TD
    A([Start]) --> B[/Input budget, expense amounts, and rates/]
    B --> C[Convert each expense to INR]
    C --> D[Add converted expenses]
    D --> E[Remaining = budget - total]
    E --> F{Remaining >= 0?}
    F -- Yes --> G[Status = Within budget]
    F -- No --> H[Status = Over budget]
    G --> I[/Display budget summary/]
    H --> I
    I --> J([End])
```

10. Campus Event Financial Planning

Problem Statement

Write a program to estimate campus event income, expenses, and net surplus or deficit from expected attendees, ticket price, sponsorship, and event costs.

Algorithm

1. Read expected attendees, ticket price, sponsorship, venue cost, food cost, equipment cost, and other costs.

2. Ticket income = attendees × ticket price.

3. Total income = ticket income + sponsorship.

4. Total expenses = venue + food + equipment + other costs.

5. Net result = total income – total expenses.

6. If net result > 0, status = Surplus; if net result < 0, status = Deficit; otherwise status = Break-even.

7. Display ticket income, total income, expenses, net result, and status. Stop.

Flowchart (visual outline)

Start ↓ Input attendance, ticket price, sponsorship, and costs ↓ Calculate ticket income and total income ↓ Calculate total expenses ↓ Calculate net result ↓ Classify as surplus, deficit, or break-even ↓ Display financial summary ↓ End

Sample Input

```
300
200
25000
20000
30000
8000
7000
```

Sample Output

```
Expected attendees: 300
Ticket income: Rs. 60000.00
Total income: Rs. 85000.00
Total expenses: Rs. 65000.00
Net result: Rs. 20000.00
Status: SURPLUS
```

Python 3 Code

```
attendees = int(input("Expected attendees: "))
ticket_price = float(input("Ticket price: Rs. "))
sponsorship = float(input("Sponsorship income: Rs. "))
venue = float(input("Venue cost: Rs. "))
food = float(input("Food cost: Rs. "))
equipment = float(input("Equipment cost: Rs. "))
other = float(input("Other costs: Rs. "))

ticket_income = attendees * ticket_price
total_income = ticket_income + sponsorship
total_expenses = venue + food + equipment + other
net = total_income - total_expenses

if net > 0:
    status = "SURPLUS"
elif net < 0:
    status = "DEFICIT"
else:
    status = "BREAK-EVEN"

print(f"Expected attendees: {attendees}")
print(f"Ticket income: Rs. {ticket_income:.2f}")
print(f"Total income: Rs. {total_income:.2f}")
print(f"Total expenses: Rs. {total_expenses:.2f}")
print(f"Net result: Rs. {net:.2f}")
print(f"Status: {status}")
```

Mermaid Flowchart Code

```
flowchart TD
    A([Start]) --> B[/Input attendance, ticket price, sponsorship, costs/]
    B --> C[Calculate ticket income]
    C --> D[Total income = ticket income + sponsorship]
    D --> E[Total expenses = sum of costs]
    E --> F[Net = total income - expenses]
    F --> G{Net > 0?}
    G -- Yes --> H[Status = SURPLUS]
    G -- No --> I{Net < 0?}
    I -- Yes --> J[Status = DEFICIT]
    I -- No --> K[Status = BREAK-EVEN]
```

```
H --> L[/Display financial summary/]
J --> L
K --> L
L --> M([End])
```